DEVORBIT — VIDEO PROOF & VIVA DEMONSTRATION GUIDE

Gaurav's Screen Recording Proof Guide

Complete Step-by-Step Script with Verified Live Cluster Endpoints (No Webhooks Needed)

Student: Gaurav (Member 3 — CI/CD & Observability)
Target Video Duration: 7 to 8 minutes
Cluster Status: 100% Verified Live (Jenkins, K8s App, Prometheus Targets UP, Grafana Live)

---

1. Verified Live Endpoints (Open These Tabs Before Recording)

Service Name	Verified Live URL	Current Live Status	Role in Recording
Jenkins CI/CD	`http://3.7.217.234:8080/job/devorbit-pipeline/`	**LIVE (HTTP 200)**	Shows 8-stage pipeline, plugins, credentials, build history.
DevOrbit App UI	`http://43.205.146.155:30080/ui/`	**LIVE (HTTP 200)**	Live React frontend running on K8s; create/update campus issues.
Prometheus	`http://43.205.146.155:30090/targets`	**LIVE (2 Pod Targets UP)**	Shows `10.42.0.30` & `10.42.0.32` being scraped every 15s.
Grafana Metrics	`http://43.205.146.155:32000`	**LIVE (HTTP 200)**	Real-time visualization of Requests/sec, Latency, and JVM heap.
AWS Console	`https://366700733051.signin.aws.amazon.com/console`	**All 3 EC2 Running (3/3 checks)**	CloudWatch Dashboard, ECR Repository, IAM, Session Manager.

⚠️ **Quick Note on the "Failed to exec spawn helper" JDK Mismatch Error:**

If you see *"Failed to exec spawn helper: exit code 1 ... JDK version mismatch"* on Jenkins, it is because Ubuntu updated the OpenJDK package in the background while Jenkins was running. The fix is a **10-second JVM restart**:

- **Method A (Browser):** Go to `http://3.7.217.234:8080/safeRestart` and click *"Yes"*.
- **Method B (AWS Console):** EC2 Instances → Select `devorbit-dev-jenkins` → Click *Connect* → *Session Manager* → Run `sudo systemctl restart jenkins`.

After restart, Java loads the updated binaries and builds proceed cleanly!

2. Screen-by-Screen Recording Walkthrough

Screen 1 (0:00 - 0:45) — Introduction & Architecture Overview

What to show: Open the terminal showing your repository at `~/DevOrbit` or the architecture diagram.

> **What to say:**
> *"Hello Ma'am, my name is Gaurav, Member 3 of the DevOrbit project. In this recording, I am demonstrating video proof for my assigned responsibilities: VPC-C infrastructure, the automated Jenkins CI/CD pipeline, AWS ECR container registry integration, and full-stack observability using CloudWatch, Prometheus, and Grafana."*

Screen 2 (0:45 - 2:00) — Jenkins Dashboard & 8 Pipeline Stages

What to show: Switch to browser tab `http://3.7.217.234:8080/job/devorbit-pipeline/`.

- Show the **8 stages all green**: Checkout, Build, Test, Package, Docker Build, Docker Push, Deploy, Verify.
- Point out the build timing and history on the left.
- Navigate to *Manage Jenkins* → *Plugins* → *Installed Plugins* → show Git, GitHub, Pipeline, JUnit, Docker Pipeline, Credentials Binding.
- Navigate to *Manage Jenkins* → *Credentials* → show `k8s-kubeconfig` (Secret file).

> **What to say:**
> *"Here is our Jenkins server running on our VPC-C instance at 3.7.217.234. You can see our 8-stage declarative pipeline. All stages pass cleanly. Jenkins uses the k8s-kubeconfig secret file to communicate with the k3s cluster over VPC peering, and all plugins were automated via our Ansible role."*

Screen 3 (2:00 - 3:00) — Jenkins Pipeline Configuration & SCM Integration

What to show: Inside the `devorbit-pipeline` job, click **Configure**.

- Scroll down to **Pipeline** section.
- Show *Definition: Pipeline script from SCM*.
- Show *SCM: Git* → Repository URL: `https://github.com/Utakarsh-Jain/DevOrbit.git`, Branch: `*/main`.
- Show *Script Path:* `jenkins/Jenkinsfile`.
- Show **Build Triggers:** *Poll SCM* (`H/5 * * * *`) which periodically checks GitHub for new commits, or manual execution via **Build Now**.

> **What to say:**
> *"In the job configuration, the pipeline is connected directly to our GitHub repository on the main branch, pointing to our Jenkinsfile. Jenkins is configured with SCM polling and manual trigger capabilities, ensuring automated builds whenever changes are detected."*

Screen 4 (3:00 - 4:30) — Live Pipeline Execution & ECR Artifact Proof

What to show: In Jenkins, click **Build Now**.

- Watch the live stage view executing:
  1. **Checkout:** Pulls code and extracts Git commit SHA (e.g., `7-d41d1e2`).
  2. **Build:** Compiles Java classes with Maven.
  3. **Test:** Executes 23 tests. Show console output: `Tests run: 23, Failures: 0, Errors: 0`.
  4. **Package:** Generates JAR.
  5. **Docker Build:** Builds 3-stage container image.
  6. **Docker Push:** Logs into AWS ECR via IAM role and pushes the image.
  7. **Deploy & Verify:** Updates Kubernetes deployment and passes health checks.
- Switch to AWS Console → **Amazon ECR** → `devorbit-app` repository.
- Show the newly pushed image tag matching the exact build number and Git SHA! Point out *Tag Immutability: Enabled* and *Vulnerability Scan: Completed*.

> **What to say:**
> *"The pipeline completed in real-time. In the console log, we see all 23 tests executed and passed. ECR authentication used the EC2 IAM role with zero hardcoded keys. In the ECR console, here is the new immutable tag matching the Git commit SHA."*

Screen 5 (4:30 - 6:00) — The Quality Failure Gate (KILLER PROOF)

What to show: Demonstrate that broken code CANNOT be deployed.

- In terminal, run `mvn test` to show all 23 tests pass first.
- Introduce a temporary assertion error in a test:

```
git switch -c demo/failure-gate
# Edit app/src/test/java/com/devorbit/IssueServiceTest.java
git commit -am "test: intentionally break test for pipeline demo"
git push origin demo/failure-gate
```

- Trigger build in Jenkins.
- Watch the build reach **Stage 3 (Test)** → turns **RED (FAILED)**!
- Highlight that **Docker Build, Docker Push, and Deploy stages were completely SKIPPED**.
- Revert the change:

```
git revert --no-edit HEAD && git push origin demo/failure-gate
```

- Show pipeline turns **GREEN** again!

> **What to say:**
> *"This is our quality failure gate. I deliberately introduced a failing assertion in a test. Notice how Jenkins caught it during the Test stage and immediately terminated the pipeline. It refused to build the Docker image or deploy to Kubernetes, protecting our production environment."*

Screen 6 (6:00 - 7:30) — Full-Stack Observability: CloudWatch, Prometheus & Grafana

What to show:

- **Live Application:** Open `http://43.205.146.155:30080/ui/` and show the working campus issue tracker.
- **AWS CloudWatch Console:** Open *Dashboards* → `devorbit-dev-dashboard`. Show CPU utilization graphs across all three EC2 instances and network throughput. Show CloudWatch Alarms for high CPU.
- **Prometheus Tab:** Open `http://43.205.146.155:30090/targets` → Show `devorbit-app` with 2 pod targets in state **UP**.
- **Grafana Tab:** Open `http://43.205.146.155:32000` → Show *DevOrbit App Metrics* dashboard with 5 panels (Requests/s, Latency, JVM Heap, 5xx rate, Live threads).
- **Generate Live Traffic:** Open terminal and run:

```
for i in {1..30}; do curl -s http://43.205.146.155:30080/api/issues > /dev/null; done
```

- Switch back to Grafana and show the request count graph spiking in real time!

> **What to say:**
> *"Here is our complete observability stack. CloudWatch monitors the infrastructure level — CPU, memory, and alarms. At the application level, Prometheus scrapes our Spring Boot Actuator endpoint every 15 seconds. And here in Grafana, as I generate live API requests in the terminal, you can immediately see the request spike reflected on the dashboard."*

Screen 7 (7:30 - 8:30) — Code Walkthrough & Git Commits Proof

What to show: In terminal or VS Code, briefly scroll through your source files:

- `jenkins/Jenkinsfile` — highlight stages, environment variables, timeout, and health retries.
- `ansible/roles/jenkins/tasks/main.yml` — highlight automated package & plugin installation.
- `terraform/security_ci.tf` & `terraform/compute_ci.tf` — highlight the VPC-C Jenkins server and Elastic IP.
- Run:

```
git log --oneline | head -20
```

Show the conventional commits documenting your CI/CD, ECR, and monitoring contributions.

> **What to say:**
> *"Everything shown in this video is fully codified in our repository under version control. This completes my demonstration of the CI/CD pipeline and monitoring infrastructure for DevOrbit. Thank you Ma'am!"*

3. Final Verification Checklist

Proof Item	Location / Command	Verified Live Status
Jenkins 8 Stages Green	`http://3.7.217.234:8080/job/devorbit-pipeline/`	Verified Live
Jenkins SCM Git Integration	Jenkins Job → Configure → Pipeline from SCM	Verified Live
Quality Gate Failure Demo	Jenkins build stopping at Stage 3 (Test)	Verified Live
ECR Image with Git SHA Tag	AWS Console → ECR → `devorbit-app` (e.g. `7-d41d1e2`)	Verified Live
CloudWatch Dashboard & CPU Alarms	AWS Console → CloudWatch → `devorbit-dev-dashboard`	Verified Live
Prometheus 2 Pod Targets UP	`http://43.205.146.155:30090/targets`	Verified Live (`10.42.0.30` & `10.42.0.32`)
Grafana Live Metrics Spike	`http://43.205.146.155:32000`	Verified Live
DevOrbit Campus App UI	`http://43.205.146.155:30080/ui/`	Verified Live (HTTP 200)